Load Balancer Health Checker

Assignment 19 | AWS Lambda + Boto3 + SNS
Step-by-step instructions | Mumbai region: ap-south-1

GitHub repository: https://github.com/thupeshkumar/lambda

Goal: check registered instances behind a given ELB and send a detailed SNS notification when any instance is unhealthy. Run the function every 10 minutes using an Amazon EventBridge rule, formerly CloudWatch Events.

Workflow: EventBridge (10 minutes) -> Lambda -> Classic ELB health API -> SNS email when unhealthy.

1. Prepare the load balancer

Use an existing **Classic Load Balancer** in ap-south-1. In EC2 > Load Balancers, record its name and verify that EC2 instances are registered and its health check is configured. This guide uses the Classic ELB API; ALB/NLB monitoring requires target groups and the elbv2 API.

You need AWS permissions to manage Lambda, IAM, SNS and EventBridge. If no load balancer exists, create a disposable test environment first; EC2 and load balancer resources incur charges.

2. Create an SNS topic and subscription

Open Amazon SNS in ap-south-1. Create a **Standard** topic named **elb-health-alerts**. Copy the topic ARN. Create an **Email** subscription with your email address and confirm it using the link in the SNS email.

3. Create the Lambda execution role

In IAM > Roles > Create role, choose AWS service and Lambda. Attach **AWSLambdaBasicExecutionRole** for CloudWatch logging. Add the inline policy below, replacing the example account ID and topic name with the actual SNS ARN. Name the role **ELBHealthCheckerRole**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "elasticloadbalancing:DescribeInstanceHealth",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:ap-south-1:123456789012:elb-health-alerts"
    }
  ]
}
```

4. Create and configure the Lambda function

In Lambda > Create function, select Author from scratch. Name it **ELBHealthChecker**, choose Python 3.12, and use the existing **ELBHealthCheckerRole**. Set the timeout to 60 seconds. Keep the function outside a VPC for this exercise so it can reach AWS service endpoints.

Under Configuration > Environment variables, add **ELB_NAME** = your Classic ELB name and **SNS_TOPIC_ARN** = the topic ARN. Set the handler to **lambda_function.lambda_handler**. Paste this code into lambda_function.py and select Deploy. Boto3 is included in the Lambda Python runtime.

```
"""Check registered Classic ELB instances and publish unhealthy-instance alerts."""
import json
import logging
import os
from datetime import datetime, timezone

import boto3

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)


def lambda_handler(event, context):
    load_balancer = os.environ["ELB_NAME"].strip()
    topic_arn = os.environ["SNS_TOPIC_ARN"].strip()
    if not load_balancer or not topic_arn:
        raise ValueError("ELB_NAME and SNS_TOPIC_ARN must be nonempty")
    elb = boto3.client("elb")
    sns = boto3.client("sns", region_name=topic_arn.split(":")[3])
    states = elb.describe_instance_health(
        LoadBalancerName=load_balancer
    )["InstanceStates"]
    unhealthy = [state for state in states if state["State"] == "OutOfService"]
    unknown = [state for state in states if state["State"] == "Unknown"]
    report = {
        "load_balancer": load_balancer,
        "region": elb.meta.region_name,
        "checked_at": datetime.now(timezone.utc).isoformat(),
        "request_id": getattr(context, "aws_request_id", None),
        "registered_count": len(states),
        "unhealthy_count": len(unhealthy),
        "unknown_count": len(unknown),
        "unhealthy_instances": unhealthy,
        "unknown_instances": unknown,
    }
    if unhealthy:
        response = sns.publish(
            TopicArn=topic_arn,
            Subject="ELB unhealthy instance alert",
            Message=json.dumps(report, indent=2),
        )
        report["sns_message_id"] = response["MessageId"]
    logger.info("ELB health report: %s", json.dumps(report))
    return report
```

5. Run a manual health check

In the Lambda Test tab, create a test event containing **{}** and select Test. The response reports registered_count, unhealthy_count and unknown_count. Open Monitor > View CloudWatch logs to inspect the report. An SNS MessageId in the response means SNS accepted the alert; confirm email delivery separately.

Classic ELB reports healthy instances as **InService** and unhealthy instances as **OutOfService**. This code alerts only for OutOfService. Unknown states and an empty instance list are logged. Each alert includes the instance ID, state, reason code, description, region, check time and counts.

6. Schedule the function every 10 minutes

Open Amazon EventBridge > Rules in ap-south-1. Create a rule named **elb-health-check-every-10-minutes** on the default event bus. Select a schedule rule and use the rate expression below. If the console recommends EventBridge Scheduler, choose the scheduled-rule option to follow this CloudWatch Events assignment.

```
rate(10 minutes)
```

Choose AWS service > Lambda function as the target and select **ELBHealthChecker**. Create and enable the rule. Verify that the target is correct. The Lambda resource policy must permit events.amazonaws.com to invoke the function, restricted to this rule ARN. The console normally adds that permission; if missing, use the command below in AWS CloudShell after substituting your account ID.

```
aws lambda add-permission \
  --region ap-south-1 \
  --function-name ELBHealthChecker \
  --statement-id AllowEventBridgeHealthCheck \
  --action lambda:InvokeFunction \
  --principal events.amazonaws.com \
  --source-arn arn:aws:events:ap-south-1:123456789012:rule/elb-health-check-every-10-minutes
```

Wait for scheduled invocations and check CloudWatch logs. Rate schedules are approximate. Notifications repeat every 10 minutes while an instance remains unhealthy; retries can produce duplicate alerts.

7. Verify an unhealthy instance

In a disposable test environment, temporarily stop the application that serves the health-check port on one registered instance. Wait until the ELB shows OutOfService. Invoke Lambda and verify the SNS email identifies the affected instance and failure details. Restore the application, wait for InService, and verify the next run sends no unhealthy alert.

Troubleshooting

AccessDenied: verify the role policy and actual topic ARN. **No email:** confirm the subscription and check the SNS MessageId. **No scheduled run:** check the enabled rule, Lambda target and invocation permission. **ELB not found:** check region and Classic ELB name. API and SNS errors appear as failed Lambda invocations.

8. Document and submit

Repository destination: https://github.com/thupeshkumar/lambda

Add the Python code, deployment template, README and this guide to an **elb-health-checker/** folder in the repository. Commit and push the files, then verify the code is visible on GitHub. Share the repository link for submission.

Status: this PDF describes the setup procedure. It does not establish that AWS resources are deployed or that code has been pushed. The earlier implementation passed five local tests and AWS template validation; GitHub publishing was blocked by credential access. No real AWS deployment screenshots are available yet.

Screenshot checklist

- Classic ELB details and registered instance health.

- SNS topic and confirmed email subscription.

- Lambda code, handler, environment variables and IAM permissions.

- Manual test result and CloudWatch health report.

- SNS email showing the unhealthy instance ID and reason.

- Enabled EventBridge rule, rate(10 minutes), and Lambda target.

Capture these screenshots after performing the steps, add them to a screenshots/ folder, and reference them in the README. Use actual results and obscure credentials or personal information before sharing.

Optional deployment template

The prepared template.yaml can create the execution role, Lambda, SNS topic, EventBridge rule and invocation permission together. Upload function.zip to an S3 bucket in ap-south-1 and deploy the template with ELBName, CodeBucket and CodeKey parameters. The email subscription still needs confirmation. See the accompanying README for PowerShell deployment commands.

Cleanup after the assignment

Disable or delete the EventBridge rule. Delete the test Lambda, SNS subscription/topic and dedicated IAM role when no longer needed. If using CloudFormation, delete its stack. Remove uploaded code artifacts and CloudWatch log groups separately. Remove disposable test ELB/EC2 resources only when you no longer need them.

Official references

AWS Classic ELB: describe_instance_health
Response fields and instance health states.

AWS EventBridge: scheduled rules and Lambda targets
Ten-minute schedule example and invocation permission.